Contents

DSA Solutions Guide - Part 1: Arrays & Strings **3**
Complete JavaScript Solutions with Detailed Explanations . . . 3

Table of Contents **3**

1. Two Sum **3**
Problem Statement . . . 3
Example . . . 3
Approach 1: Brute Force - O(n^2) . . . 4
Explanation . . . 4
Approach 2: Hash Map - O(n) [OPTIMAL] . . . 4
Explanation . . . 4
Visual Walkthrough . . . 4
Why This Works . . . 5

2. Best Time to Buy and Sell Stock **5**
Problem Statement . . . 5
Example . . . 5
Approach: One Pass - O(n) . . . 5
Explanation . . . 5
Visual Walkthrough . . . 5
Key Insight . . . 6

3. Contains Duplicate **6**
Problem Statement . . . 6
Example . . . 6
Approach 1: Sorting - O(n log n) . . . 6
Approach 2: Hash Set - O(n) [OPTIMAL] . . . 7
Explanation . . . 7
One-Liner Solution . . . 7

4. Maximum Subarray (Kadane's Algorithm) **7**
Problem Statement . . . 7
Example . . . 7
Kadane's Algorithm - O(n) . . . 7
Explanation . . . 7
Visual Walkthrough . . . 7
Extended: Return the Actual Subarray . . . 8

5. Move Zeroes **9**
Problem Statement . . . 9
Example . . . 9
Two Pointers Approach - O(n) . . . 9
Explanation . . . 9
Optimized: Single Pass with Swap . . . 9

6. Valid Anagram **10**
Problem Statement . . . 10
Example . . . 10
Approach 1: Sorting - O(n log n) . . . 10
Approach 2: Character Frequency - O(n) [OPTIMAL] . . . 10
Explanation . . . 10
Using Array (For Lowercase Letters Only) . . . 10

7. Valid Palindrome **11**
Problem Statement . . . 11

Example . . . 11
Two Pointers Approach - O(n) . . . 11
Cleaner Version with Regex . . . 12

8. 3Sum **12**
Problem Statement . . . 12
Example . . . 12
Approach: Sort + Two Pointers - O(n^2) . . . 12
Explanation . . . 12
Visual Walkthrough . . . 12

9. Container With Most Water **14**
Problem Statement . . . 14
Example . . . 14
Two Pointers Approach - O(n) . . . 14
Explanation . . . 14
Why Moving Smaller Pointer Works . . . 15

10. Product of Array Except Self **15**
Problem Statement . . . 15
Example . . . 15
Approach: Prefix and Suffix Products - O(n) . . . 15
Explanation . . . 15
Visual Walkthrough . . . 15

11. Longest Consecutive Sequence **16**
Problem Statement . . . 16
Example . . . 16
Hash Set Approach - O(n) . . . 16
Explanation . . . 16
Why This is O(n) . . . 16

12. Group Anagrams **17**
Problem Statement . . . 17
Example . . . 17
Approach: Sorted String as Key - O(n * k log k) . . . 17
Explanation . . . 17
Optimized: Character Count as Key - O(n * k) . . . 18

13. Longest Substring Without Repeating Characters **18**
Problem Statement . . . 18
Example . . . 18
Sliding Window Approach - O(n) . . . 18
Explanation . . . 18
Visual Walkthrough . . . 18
Using Set (Alternative) . . . 19

14. Merge Intervals **20**
Problem Statement . . . 20
Example . . . 20
Approach: Sort and Merge - O(n log n) . . . 20
Explanation . . . 20
Visual Walkthrough . . . 20

15. Trapping Rain Water **21**
Problem Statement . . . 21
Example . . . 21
Approach 1: Two Pointers - O(n) Time, O(1) Space [OPTIMAL] . . . 21

Explanation . . . 21
Approach 2: Precompute Max Arrays - O(n) Time, O(n) Space . . . 22
Approach 3: Monotonic Stack - O(n) . . . 22

16. Minimum Window Substring **23**
Problem Statement . . . 23
Example . . . 23
Sliding Window Approach - O(n) . . . 23
Explanation . . . 23

17. Sliding Window Maximum **24**
Problem Statement . . . 24
Example . . . 24
Approach: Monotonic Decreasing Deque - O(n) . . . 24
Explanation . . . 24
Visual Walkthrough . . . 25

Summary: Key Patterns for Arrays & Strings **25**

DSA Solutions Guide - Part 1: Arrays & Strings

Complete JavaScript Solutions with Detailed Explanations

Table of Contents

1. Two Sum
2. Best Time to Buy and Sell Stock
3. Contains Duplicate
4. Maximum Subarray (Kadane's Algorithm)
5. Move Zeroes
6. Valid Anagram
7. Valid Palindrome
8. 3Sum
9. Container With Most Water
10. Product of Array Except Self
11. Longest Consecutive Sequence
12. Group Anagrams
13. Longest Substring Without Repeating Characters
14. Merge Intervals
15. Trapping Rain Water
16. Minimum Window Substring
17. Sliding Window Maximum

---

1. Two Sum

Problem Statement

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution.

Example

```
Input: nums = [2, 7, 11, 15], target = 9
```

```
Output: [0, 1]
Explanation: nums[0] + nums[1] = 2 + 7 = 9
```

Approach 1: Brute Force - O(n^2)

Explanation

Check every pair of numbers to see if they sum to target.

```javascript
function twoSumBruteForce(nums, target) {
    const n = nums.length;

    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            if (nums[i] + nums[j] === target) {
                return [i, j];
            }
        }
    }

    return []; // No solution found
}

// Time: O(n^2) - nested loops
// Space: O(1) - no extra space
```

Approach 2: Hash Map - O(n) [OPTIMAL]

Explanation

1. Create a hash map to store each number and its index
2. For each number, calculate its complement (target - current number)
3. Check if complement exists in the map
4. If yes, we found our pair!

Visual Walkthrough

```
nums = [2, 7, 11, 15], target = 9

Step 1: num = 2, complement = 9 - 2 = 7
        Map: {} -> 7 not in map
        Add 2 to map: {2: 0}

Step 2: num = 7, complement = 9 - 7 = 2
        Map: {2: 0} -> 2 IS in map!
        Return [map[2], current_index] = [0, 1]
```

```javascript
function twoSum(nums, target) {
    // Hash map to store number -> index mapping
    const numToIndex = new Map();

    for (let i = 0; i < nums.length; i++) {
        const complement = target - nums[i];

        // Check if complement exists in our map
        if (numToIndex.has(complement)) {
            return [numToIndex.get(complement), i];
        }
```

```
        // Store current number with its index
        numToIndex.set(nums[i], i);
    }

    return []; // No solution found
}

// Time: O(n) - single pass through array
// Space: O(n) - hash map stores up to n elements
```

Why This Works

- We traverse the array once
- For each element, we check if we've seen a number that would complete our sum
- The hash map gives us O(1) lookup time
- We store elements as we go, so we never match an element with itself

---

2. Best Time to Buy and Sell Stock

Problem Statement

Given an array `prices` where `prices[i]` is the price of a stock on day i, find the maximum profit from one transaction (buy one day, sell another later day).

Example

```
Input: prices = [7, 1, 5, 3, 6, 4]
Output: 5
Explanation: Buy on day 2 (price = 1), sell on day 5 (price = 6), profit = 6 - 1 = 5
```

Approach: One Pass - O(n)

Explanation

1. Track the minimum price seen so far
2. At each day, calculate potential profit if we sell today
3. Keep track of maximum profit

Visual Walkthrough

```
prices = [7, 1, 5, 3, 6, 4]

Day 0: price=7, minPrice=7, profit=0, maxProfit=0
Day 1: price=1, minPrice=1, profit=0, maxProfit=0  (new minimum!)
Day 2: price=5, minPrice=1, profit=4, maxProfit=4
Day 3: price=3, minPrice=1, profit=2, maxProfit=4
Day 4: price=6, minPrice=1, profit=5, maxProfit=5  (new max!)
Day 5: price=4, minPrice=1, profit=3, maxProfit=5

Return 5
```

```
function maxProfit(prices) {
    if (prices.length < 2) return 0;

    let minPrice = prices[0];  // Minimum price seen so far
    let maxProfit = 0;         // Maximum profit possible
```

```
    for (let i = 1; i < prices.length; i++) {
        // Calculate profit if we sell today
        const currentProfit = prices[i] - minPrice;

        // Update maximum profit
        maxProfit = Math.max(maxProfit, currentProfit);

        // Update minimum price
        minPrice = Math.min(minPrice, prices[i]);
    }

    return maxProfit;
}

// Time: O(n) - single pass
// Space: O(1) - only two variables
```

Key Insight

We want to buy at the lowest point BEFORE we sell at a high point. By tracking the minimum price as we go, we ensure we're always considering the best possible buy price for any future sell day.

---

3. Contains Duplicate

Problem Statement

Given an integer array `nums`, return `true` if any value appears at least twice.

Example

```
Input: nums = [1, 2, 3, 1]
Output: true

Input: nums = [1, 2, 3, 4]
Output: false
```

Approach 1: Sorting - O(n log n)

```
function containsDuplicateSorting(nums) {
    // Sort the array
    nums.sort((a, b) => a - b);

    // Check adjacent elements
    for (let i = 1; i < nums.length; i++) {
        if (nums[i] === nums[i - 1]) {
            return true;
        }
    }

    return false;
}

// Time: O(n log n) - sorting dominates
// Space: O(1) or O(n) depending on sort implementation
```

Approach 2: Hash Set - O(n) [OPTIMAL]

Explanation

Use a Set to track seen numbers. If we try to add a number that already exists, we found a duplicate.

```
function containsDuplicate(nums) {
    const seen = new Set();

    for (const num of nums) {
        if (seen.has(num)) {
            return true;  // Duplicate found!
        }
        seen.add(num);
    }

    return false;
}

// Time: O(n) - single pass
// Space: O(n) - set can hold up to n elements
```

One-Liner Solution

```
function containsDuplicateOneLiner(nums) {
    return new Set(nums).size !== nums.length;
}
```

---

4. Maximum Subarray (Kadane's Algorithm)

Problem Statement

Find the contiguous subarray with the largest sum.

Example

```
Input: nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: Subarray [4, -1, 2, 1] has the largest sum = 6
```

Kadane's Algorithm - O(n)

Explanation

The key insight is: at each position, we decide whether to: 1. Start a new subarray from current element, OR 2. Extend the previous subarray

We extend if adding current element to previous sum is better than starting fresh.

Visual Walkthrough

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]

i=0: num=-2, currentSum=max(-2, 0+(-2))=-2, maxSum=-2
i=1: num=1,  currentSum=max(1, -2+1)=1,     maxSum=1   (start fresh!)
i=2: num=-3, currentSum=max(-3, 1+(-3))=-2, maxSum=1
i=3: num=4,  currentSum=max(4, -2+4)=4,     maxSum=4   (start fresh!)
i=4: num=-1, currentSum=max(-1, 4+(-1))=3,  maxSum=4
```

```
i=5: num=2,  currentSum=max(2, 3+2)=5,      maxSum=5
i=6: num=1,  currentSum=max(1, 5+1)=6,      maxSum=6   (new max!)
i=7: num=-5, currentSum=max(-5, 6+(-5))=1,  maxSum=6
i=8: num=4,  currentSum=max(4, 1+4)=5,      maxSum=6

Return 6
```

```javascript
function maxSubArray(nums) {
    if (nums.length === 0) return 0;

    let currentSum = nums[0];  // Current subarray sum
    let maxSum = nums[0];      // Maximum sum found

    for (let i = 1; i < nums.length; i++) {
        // Either start new subarray or extend previous
        currentSum = Math.max(nums[i], currentSum + nums[i]);

        // Update maximum
        maxSum = Math.max(maxSum, currentSum);
    }

    return maxSum;
}

// Time: O(n) - single pass
// Space: O(1) - constant extra space
```

Extended: Return the Actual Subarray

```javascript
function maxSubArrayWithIndices(nums) {
    let currentSum = nums[0];
    let maxSum = nums[0];
    let start = 0, end = 0, tempStart = 0;

    for (let i = 1; i < nums.length; i++) {
        if (nums[i] > currentSum + nums[i]) {
            currentSum = nums[i];
            tempStart = i;  // New potential start
        } else {
            currentSum = currentSum + nums[i];
        }

        if (currentSum > maxSum) {
            maxSum = currentSum;
            start = tempStart;
            end = i;
        }
    }

    return {
        maxSum,
        subarray: nums.slice(start, end + 1)
    };
}
```

---

5. Move Zeroes

Problem Statement

Move all zeros to the end while maintaining relative order of non-zero elements. Do this in-place.

Example

```
Input: nums = [0, 1, 0, 3, 12]
Output: [1, 3, 12, 0, 0]
```

Two Pointers Approach - O(n)

Explanation

Use two pointers: - `writePointer`: position where next non-zero should be written - Loop through array, when we find non-zero, write it to writePointer position

```javascript
function moveZeroes(nums) {
    let writePointer = 0;  // Where to write next non-zero

    // First pass: move all non-zeros to front
    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {
            nums[writePointer] = nums[i];
            writePointer++;
        }
    }

    // Second pass: fill remaining positions with zeros
    while (writePointer < nums.length) {
        nums[writePointer] = 0;
        writePointer++;
    }

    return nums;
}

// Time: O(n) - two passes
// Space: O(1) - in-place
```

Optimized: Single Pass with Swap

```javascript
function moveZeroesOptimal(nums) {
    let writePointer = 0;

    for (let i = 0; i < nums.length; i++) {
        if (nums[i] !== 0) {
            // Swap current element with position at writePointer
            [nums[writePointer], nums[i]] = [nums[i], nums[writePointer]];
            writePointer++;
        }
    }

    return nums;
}
```

---

6. Valid Anagram

Problem Statement

Determine if two strings are anagrams (contain same characters with same frequencies).

Example

```
Input: s = "anagram", t = "nagaram"
Output: true

Input: s = "rat", t = "car"
Output: false
```

Approach 1: Sorting - O(n log n)

```javascript
function isAnagramSorting(s, t) {
    if (s.length !== t.length) return false;

    return s.split('').sort().join('') === t.split('').sort().join('');
}
```

Approach 2: Character Frequency - O(n) [OPTIMAL]

Explanation

Count frequency of each character in both strings and compare.

```javascript
function isAnagram(s, t) {
    if (s.length !== t.length) return false;

    const charCount = new Map();

    // Count characters in s
    for (const char of s) {
        charCount.set(char, (charCount.get(char) || 0) + 1);
    }

    // Subtract counts using t
    for (const char of t) {
        if (!charCount.has(char)) return false;

        charCount.set(char, charCount.get(char) - 1);

        if (charCount.get(char) < 0) return false;
    }

    return true;
}

// Time: O(n) - two passes through strings
// Space: O(1) - at most 26 lowercase letters (constant)
```

Using Array (For Lowercase Letters Only)

```javascript
function isAnagramArray(s, t) {
    if (s.length !== t.length) return false;

    const count = new Array(26).fill(0);
```

```
    const aCode = 'a'.charCodeAt(0);

    for (let i = 0; i < s.length; i++) {
        count[s.charCodeAt(i) - aCode]++;
        count[t.charCodeAt(i) - aCode]--;
    }

    return count.every(c => c === 0);
}
```

---

7. Valid Palindrome

Problem Statement

Check if a string is a palindrome, considering only alphanumeric characters and ignoring case.

Example

```
Input: s = "A man, a plan, a canal: Panama"
Output: true (reads "amanaplanacanalpanama" forwards and backwards)
```

Two Pointers Approach - O(n)

```
function isPalindrome(s) {
    // Helper function to check if character is alphanumeric
    const isAlphanumeric = (char) => {
        const code = char.charCodeAt(0);
        return (code >= 48 && code <= 57) ||  // 0-9
               (code >= 65 && code <= 90) ||  // A-Z
               (code >= 97 && code <= 122);   // a-z
    };

    let left = 0;
    let right = s.length - 1;

    while (left < right) {
        // Skip non-alphanumeric characters from left
        while (left < right && !isAlphanumeric(s[left])) {
            left++;
        }

        // Skip non-alphanumeric characters from right
        while (left < right && !isAlphanumeric(s[right])) {
            right--;
        }

        // Compare characters (case-insensitive)
        if (s[left].toLowerCase() !== s[right].toLowerCase()) {
            return false;
        }

        left++;
        right--;
    }
```

```
    return true;
}

// Time: O(n) - each character visited at most twice
// Space: O(1) - no extra space
```

Cleaner Version with Regex

```
function isPalindromeClean(s) {
    // Remove non-alphanumeric and convert to lowercase
    const cleaned = s.replace(/[^a-zA-Z0-9]/g, '').toLowerCase();

    // Check if palindrome
    let left = 0;
    let right = cleaned.length - 1;

    while (left < right) {
        if (cleaned[left] !== cleaned[right]) {
            return false;
        }
        left++;
        right--;
    }

    return true;
}

// Note: This uses O(n) extra space for cleaned string
```

---

8. 3Sum

Problem Statement

Find all unique triplets in the array that sum to zero.

Example

```
Input: nums = [-1, 0, 1, 2, -1, -4]
Output: [[-1, -1, 2], [-1, 0, 1]]
```

Approach: Sort + Two Pointers - O(n^2)

Explanation

1. Sort the array
2. Fix one number (iterate through array)
3. Use two pointers to find pairs that sum to negative of fixed number
4. Skip duplicates to avoid duplicate triplets

Visual Walkthrough

```
nums = [-1, 0, 1, 2, -1, -4]
After sort: [-4, -1, -1, 0, 1, 2]

i=0, nums[i]=-4, target=4
    left=1, right=5: -1+2=1 < 4, left++
```

```
    left=2, right=5: -1+2=1 < 4, left++
    left=3, right=5: 0+2=2 < 4, left++
    left=4, right=5: 1+2=3 < 4, left++
    left >= right, done

i=1, nums[i]=-1, target=1
    left=2, right=5: -1+2=1 == 1, FOUND [-1,-1,2]!
    left++, right--
    left=3, right=4: 0+1=1 == 1, FOUND [-1,0,1]!
    left++, right--
    left >= right, done

i=2, nums[i]=-1, SKIP (duplicate of i=1)

i=3, nums[i]=0, target=0
    left=4, right=5: 1+2=3 > 0, right--
    left >= right, done

Result: [[-1,-1,2], [-1,0,1]]
```

```
function threeSum(nums) {
    const result = [];

    // Sort the array
    nums.sort((a, b) => a - b);

    for (let i = 0; i < nums.length - 2; i++) {
        // Skip duplicates for first number
        if (i > 0 && nums[i] === nums[i - 1]) continue;

        // Early termination: if smallest is positive, no solution possible
        if (nums[i] > 0) break;

        const target = -nums[i];
        let left = i + 1;
        let right = nums.length - 1;

        while (left < right) {
            const sum = nums[left] + nums[right];

            if (sum === target) {
                result.push([nums[i], nums[left], nums[right]]);

                // Skip duplicates for second number
                while (left < right && nums[left] === nums[left + 1]) left++;
                // Skip duplicates for third number
                while (left < right && nums[right] === nums[right - 1]) right--;

                left++;
                right--;
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
    }
```

```
    return result;
}

// Time: O(n^2) - outer loop O(n), two pointers O(n)
// Space: O(1) - excluding output array (or O(n) for sorting)
```

---

9. Container With Most Water

Problem Statement

Given n vertical lines at positions with heights `height[i]`, find two lines that together with x-axis form a container holding the most water.

Example

```
Input: height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output: 49
Explanation: Lines at index 1 (height=8) and index 8 (height=7)
             Width = 8 - 1 = 7, Height = min(8, 7) = 7
             Area = 7 * 7 = 49
```

Two Pointers Approach - O(n)

Explanation

1. Start with widest container (left at 0, right at end)
2. Calculate area: width * min(height[left], height[right])
3. Move the pointer with smaller height inward (greedy choice)
4. Why? Moving the smaller one might find a taller line; moving the taller one can only decrease area

```
function maxArea(height) {
    let left = 0;
    let right = height.length - 1;
    let maxWater = 0;

    while (left < right) {
        // Calculate current area
        const width = right - left;
        const minHeight = Math.min(height[left], height[right]);
        const area = width * minHeight;

        maxWater = Math.max(maxWater, area);

        // Move pointer with smaller height
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return maxWater;
}
```

```
// Time: O(n) - single pass
// Space: O(1) - constant extra space
```

Why Moving Smaller Pointer Works

- Current area is limited by the shorter line
- If we move the taller line, width decreases and height can only stay same or decrease
- If we move the shorter line, we might find a taller line that increases the limiting height
- This greedy approach guarantees we don't miss the optimal solution

---

10. Product of Array Except Self

Problem Statement

Return an array where each element is the product of all other elements. Do not use division.

Example

```
Input: nums = [1, 2, 3, 4]
Output: [24, 12, 8, 6]
Explanation:
  result[0] = 2*3*4 = 24
  result[1] = 1*3*4 = 12
  result[2] = 1*2*4 = 8
  result[3] = 1*2*3 = 6
```

Approach: Prefix and Suffix Products - O(n)

Explanation

For each index i, the result is: - Product of all elements to the LEFT of i, times - Product of all elements to the RIGHT of i

We can compute this in two passes.

Visual Walkthrough

```
nums = [1, 2, 3, 4]

Left products (prefix):
  left[0] = 1 (nothing to left)
  left[1] = 1
  left[2] = 1 * 2 = 2
  left[3] = 1 * 2 * 3 = 6

Right products (suffix):
  right[3] = 1 (nothing to right)
  right[2] = 4
  right[1] = 4 * 3 = 12
  right[0] = 4 * 3 * 2 = 24

Result = left * right:
  result[0] = 1 * 24 = 24
  result[1] = 1 * 12 = 12
  result[2] = 2 * 4 = 8
  result[3] = 6 * 1 = 6
```

```
function productExceptSelf(nums) {
    const n = nums.length;
    const result = new Array(n);

    // First pass: calculate left products
    result[0] = 1;
    for (let i = 1; i < n; i++) {
        result[i] = result[i - 1] * nums[i - 1];
    }

    // Second pass: multiply by right products
    let rightProduct = 1;
    for (let i = n - 1; i >= 0; i--) {
        result[i] = result[i] * rightProduct;
        rightProduct *= nums[i];
    }

    return result;
}

// Time: O(n) - two passes
// Space: O(1) - output array doesn't count as extra space
```

---

11. Longest Consecutive Sequence

Problem Statement

Find the length of the longest consecutive elements sequence in O(n) time.

Example

```
Input: nums = [100, 4, 200, 1, 3, 2]
Output: 4
Explanation: Longest consecutive sequence is [1, 2, 3, 4]
```

Hash Set Approach - O(n)

Explanation

1. Put all numbers in a Set
2. For each number, check if it's the START of a sequence (num-1 not in set)
3. If it's a start, count consecutive numbers

Why This is O(n)

Each number is visited at most twice: once when checking if it's a start, once when counting sequence.

```
function longestConsecutive(nums) {
    if (nums.length === 0) return 0;

    const numSet = new Set(nums);
    let maxLength = 0;

    for (const num of numSet) {
        // Only start counting if this is the beginning of a sequence
        if (!numSet.has(num - 1)) {
```

```
            let currentNum = num;
            let currentLength = 1;

            // Count consecutive numbers
            while (numSet.has(currentNum + 1)) {
                currentNum++;
                currentLength++;
            }

            maxLength = Math.max(maxLength, currentLength);
        }
    }

    return maxLength;
}

// Time: O(n) - each number processed at most twice
// Space: O(n) - hash set
```

---

12. Group Anagrams

Problem Statement

Group strings that are anagrams of each other.

Example

```
Input: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
Output: [["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]
```

Approach: Sorted String as Key - O(n * k log k)

Explanation

Two strings are anagrams if they have the same sorted characters. Use sorted string as key in a hash map.

```
function groupAnagrams(strs) {
    const groups = new Map();

    for (const str of strs) {
        // Create key by sorting characters
        const key = str.split('').sort().join('');

        if (!groups.has(key)) {
            groups.set(key, []);
        }
        groups.get(key).push(str);
    }

    return Array.from(groups.values());
}

// Time: O(n * k log k) where n = number of strings, k = max string length
// Space: O(n * k) - storing all strings
```

Optimized: Character Count as Key - O(n * k)

```javascript
function groupAnagramsOptimal(strs) {
    const groups = new Map();

    for (const str of strs) {
        // Create key from character counts
        const count = new Array(26).fill(0);
        for (const char of str) {
            count[char.charCodeAt(0) - 97]++;
        }
        const key = count.join('#');  // e.g., "1#0#0#...#1#0"

        if (!groups.has(key)) {
            groups.set(key, []);
        }
        groups.get(key).push(str);
    }

    return Array.from(groups.values());
}

// Time: O(n * k) - no sorting needed
// Space: O(n * k)
```

---

13. Longest Substring Without Repeating Characters

Problem Statement

Find the length of the longest substring without repeating characters.

Example

```
Input: s = "abcabcbb"
Output: 3
Explanation: "abc" is the longest substring without repeating characters
```

Sliding Window Approach - O(n)

Explanation

Use a sliding window with two pointers. Expand right pointer, and when we see a duplicate, shrink from left until no duplicate.

Visual Walkthrough

```
s = "abcabcbb"

Window: ""          left=0, right=0
        "a"         right=0, seen={a:0}, length=1
        "ab"        right=1, seen={a:0,b:1}, length=2
        "abc"       right=2, seen={a:0,b:1,c:2}, length=3

right=3, char='a' is in seen at index 0
        Move left to max(0, 0+1)=1
        "bca"       seen={a:3,b:1,c:2}, length=3
```

```
right=4, char='b' is in seen at index 1
        Move left to max(1, 1+1)=2
        "cab"       seen={a:3,b:4,c:2}, length=3

... and so on

Max length = 3
```

```javascript
function lengthOfLongestSubstring(s) {
    const charIndex = new Map();  // char -> last seen index
    let maxLength = 0;
    let left = 0;

    for (let right = 0; right < s.length; right++) {
        const char = s[right];

        // If character was seen and is within current window
        if (charIndex.has(char) && charIndex.get(char) >= left) {
            // Move left pointer past the previous occurrence
            left = charIndex.get(char) + 1;
        }

        // Update last seen index
        charIndex.set(char, right);

        // Update max length
        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}

// Time: O(n) - single pass
// Space: O(min(m, n)) where m = charset size
```

Using Set (Alternative)

```javascript
function lengthOfLongestSubstringSet(s) {
    const seen = new Set();
    let maxLength = 0;
    let left = 0;

    for (let right = 0; right < s.length; right++) {
        // Remove characters from left until no duplicate
        while (seen.has(s[right])) {
            seen.delete(s[left]);
            left++;
        }

        seen.add(s[right]);
        maxLength = Math.max(maxLength, right - left + 1);
    }

    return maxLength;
}
```

---

14. Merge Intervals

Problem Statement

Merge all overlapping intervals.

Example

```
Input: intervals = [[1,3], [2,6], [8,10], [15,18]]
Output: [[1,6], [8,10], [15,18]]
Explanation: [1,3] and [2,6] overlap, merge into [1,6]
```

Approach: Sort and Merge - O(n log n)

Explanation

1. Sort intervals by start time
2. Iterate through, merging overlapping intervals
3. Two intervals overlap if current.start <= previous.end

```javascript
function merge(intervals) {
    if (intervals.length <= 1) return intervals;

    // Sort by start time
    intervals.sort((a, b) => a[0] - b[0]);

    const result = [intervals[0]];

    for (let i = 1; i < intervals.length; i++) {
        const current = intervals[i];
        const lastMerged = result[result.length - 1];

        // Check if overlapping
        if (current[0] <= lastMerged[1]) {
            // Merge: extend end time if needed
            lastMerged[1] = Math.max(lastMerged[1], current[1]);
        } else {
            // No overlap: add as new interval
            result.push(current);
        }
    }

    return result;
}

// Time: O(n log n) - sorting dominates
// Space: O(n) - result array (or O(log n) for sorting)
```

Visual Walkthrough

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
After sort: [[1,3], [2,6], [8,10], [15,18]] (already sorted)

result = [[1,3]]

i=1: [2,6], last=[1,3]
     2 <= 3? YES, overlap!
     Merge: [1, max(3,6)] = [1,6]
```

```
     result = [[1,6]]

i=2: [8,10], last=[1,6]
     8 <= 6? NO, no overlap
     result = [[1,6], [8,10]]

i=3: [15,18], last=[8,10]
     15 <= 10? NO, no overlap
     result = [[1,6], [8,10], [15,18]]
```

---

15. Trapping Rain Water

Problem Statement

Given n non-negative integers representing elevation map, compute how much water it can trap after raining.

Example

```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
```

Approach 1: Two Pointers - O(n) Time, O(1) Space [OPTIMAL]

Explanation

Water trapped at index i = min(maxLeft, maxRight) - height[i]

Key insight: We only need to know the minimum of max heights on both sides. If left max is smaller, we know water level at left position. Vice versa for right.

```javascript
function trap(height) {
    if (height.length < 3) return 0;

    let left = 0;
    let right = height.length - 1;
    let leftMax = 0;
    let rightMax = 0;
    let water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            // Left side is limiting factor
            if (height[left] >= leftMax) {
                leftMax = height[left];
            } else {
                water += leftMax - height[left];
            }
            left++;
        } else {
            // Right side is limiting factor
            if (height[right] >= rightMax) {
                rightMax = height[right];
            } else {
                water += rightMax - height[right];
            }
            right--;
```

```
        }
    }

    return water;
}

// Time: O(n) - single pass
// Space: O(1) - constant extra space
```

Approach 2: Precompute Max Arrays - O(n) Time, O(n) Space

```
function trapWithArrays(height) {
    const n = height.length;
    if (n < 3) return 0;

    // Compute max height from left for each position
    const leftMax = new Array(n);
    leftMax[0] = height[0];
    for (let i = 1; i < n; i++) {
        leftMax[i] = Math.max(leftMax[i - 1], height[i]);
    }

    // Compute max height from right for each position
    const rightMax = new Array(n);
    rightMax[n - 1] = height[n - 1];
    for (let i = n - 2; i >= 0; i--) {
        rightMax[i] = Math.max(rightMax[i + 1], height[i]);
    }

    // Calculate water at each position
    let water = 0;
    for (let i = 0; i < n; i++) {
        water += Math.min(leftMax[i], rightMax[i]) - height[i];
    }

    return water;
}
```

Approach 3: Monotonic Stack - O(n)

```
function trapStack(height) {
    const stack = [];  // Store indices
    let water = 0;

    for (let i = 0; i < height.length; i++) {
        while (stack.length > 0 && height[i] > height[stack[stack.length - 1]]) {
            const bottom = stack.pop();

            if (stack.length === 0) break;

            const left = stack[stack.length - 1];
            const width = i - left - 1;
            const boundedHeight = Math.min(height[left], height[i]) - height[bottom];

            water += width * boundedHeight;
        }
        stack.push(i);
```

```
    }

    return water;
}
```

---

16. Minimum Window Substring

Problem Statement

Find the minimum window substring of s such that every character in t (including duplicates) is included.

Example

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
```

Sliding Window Approach - O(n)

Explanation

1. Count required characters from t
2. Expand window (right pointer) until all characters found
3. Contract window (left pointer) while maintaining validity
4. Track minimum window

```
function minWindow(s, t) {
    if (s.length < t.length) return "";

    // Count required characters
    const required = new Map();
    for (const char of t) {
        required.set(char, (required.get(char) || 0) + 1);
    }

    const windowCounts = new Map();
    let have = 0;              // Characters with sufficient count
    const need = required.size; // Unique characters needed

    let minLength = Infinity;
    let minStart = 0;
    let left = 0;

    for (let right = 0; right < s.length; right++) {
        const char = s[right];

        // Add to window
        windowCounts.set(char, (windowCounts.get(char) || 0) + 1);

        // Check if this character's requirement is now satisfied
        if (required.has(char) && windowCounts.get(char) === required.get(char)) {
            have++;
        }

        // Try to contract window while it's valid
        while (have === need) {
            // Update minimum
```

```
            if (right - left + 1 < minLength) {
                minLength = right - left + 1;
                minStart = left;
            }

            // Remove leftmost character
            const leftChar = s[left];
            windowCounts.set(leftChar, windowCounts.get(leftChar) - 1);

            if (required.has(leftChar) &&
                windowCounts.get(leftChar) < required.get(leftChar)) {
                have--;
            }

            left++;
        }
    }

    return minLength === Infinity ? "" : s.substring(minStart, minStart + minLength);
}

// Time: O(n + m) where n = s.length, m = t.length
// Space: O(m) for the maps
```

---

17. Sliding Window Maximum

Problem Statement

Return the max sliding window for each position with window size k.

Example

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
```

Approach: Monotonic Decreasing Deque - O(n)

Explanation

Maintain a deque of indices where values are in decreasing order. The front of deque is always the maximum of current window.

```
function maxSlidingWindow(nums, k) {
    const result = [];
    const deque = [];  // Store indices, values in decreasing order

    for (let i = 0; i < nums.length; i++) {
        // Remove indices outside current window
        while (deque.length > 0 && deque[0] < i - k + 1) {
            deque.shift();
        }

        // Remove indices with smaller values (they'll never be max)
        while (deque.length > 0 && nums[deque[deque.length - 1]] < nums[i]) {
            deque.pop();
        }
```

```
        deque.push(i);

        // Add to result once we have a full window
        if (i >= k - 1) {
            result.push(nums[deque[0]]);
        }
    }

    return result;
}

// Time: O(n) - each element added and removed at most once
// Space: O(k) - deque size bounded by window size
```

Visual Walkthrough

```
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

i=0: num=1,  deque=[0]                (add index 0)
i=1: num=3,  deque=[1]                (1 < 3, remove 0, add 1)
i=2: num=-1, deque=[1,2], result=[3] (add 2, window full, max=nums[1]=3)
i=3: num=-3, deque=[1,2,3], result=[3,3] (add 3, max=nums[1]=3)
i=4: num=5,  deque=[4], result=[3,3,5]   (5 > all, clear deque, max=5)
i=5: num=3,  deque=[4,5], result=[3,3,5,5] (add 5, max=nums[4]=5)
i=6: num=6,  deque=[6], result=[3,3,5,5,6] (6 > all, max=6)
i=7: num=7,  deque=[7], result=[3,3,5,5,6,7] (7 > all, max=7)
```

Summary: Key Patterns for Arrays & Strings

Pattern	When to Use	Examples
Hash Map	Need O(1) lookup, counting	Two Sum, Group Anagrams
Two Pointers	Sorted array, opposite ends	3Sum, Container
Sliding Window	Subarray/substring problems	Longest Substring, Min Window
Prefix Sum	Range sum queries	Subarray Sum Equals K
Kadane's	Maximum subarray	Max Subarray
Sort + Search	Order matters	Merge Intervals
Monotonic Stack/Deque	Next greater, sliding max	Trapping Rain Water

---

Continue to Part 2: Linked Lists & Stack/Queue Solutions